

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 2

Operating Systems

Meet the OS You Will Build

Track B: build your own small OS (xv6)

FAF-241, Ciur Alexandrina

Checked by:
Reitman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

1 Scopul Lucrării

Familiarizarea cu sistemul de operare educațional **xv6** (o reimplementare a Unix Version 6 pe arhitectura RISC-V) rulat în emulatorul QEMU. Laboratorul implică compilarea nucleului, explorarea mediului de execuție, analiza codului sursă din directoarele `kernel/` și `user/`, precum și implementarea primului program utilitar în spațiul utilizator: `sleep.c`.

2 Pregătirea Mediului de Lucru

Sarcina: Înainte de început

Lucrul se desfășoară pe mașina virtuală Linux, unde cross-compilatorul `riscv64` și QEMU sunt deja instalate. Se accesează copia cursului a xv6 (sau se clonează depozitul):

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls    # kernel/ user/ Makefile README ...
```

3 Part 1 & 2: Rulare și Explorare xv6

Sarcina Part 1: Build it and boot it

1. Compilarea nucleului și a programelor utilizator, apoi lansarea xv6 în emulator cu `make qemu`, până la apariția prompt-ului shell-ului (`init: starting sh`). Ieșirea din QEMU se face cu `Ctrl-A`, apoi `X`.

Sarcina Part 2: Use xv6 as the Unix it is

La prompt-ul `$` din xv6 se rulează programele integrate: `ls`, `cat README`, `echo hello xv6`, `ls | grep c`, `wc README` și, opțional, `usertests -q`.

Observe: (1) Enumerați trei programe incluse în xv6. (2) Ați folosit un pipe (`|`) în xv6. Ce două funcționalități ale SO trebuie să existe pentru ca un pipe să funcționeze? (3) Într-o propoziție: cum se compară shell-ul xv6 cu shell-ul Linux din Lab 1?

3.1 Comenzi rulate în interiorul nucleului xv6

În cadrul sesiunii interactive din consola xv6 au fost rulate utilitarele de bază integrate:

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c
$ wc README
```

3.2 Observații (Part 2)

1. **Trei programe incluse cu xv6:** `ls`, `cat`, `echo` (precum și `grep`, `wc`, `sh`).

2. **Două funcționalități esențiale ale SO pentru funcționarea conductelor (*pipes*):**

   - Suportul pentru procese multiple și execuție concurentă (crearea proceselor prin `fork` și planificarea lor).
   - Mecanismul de comunicație inter-procese (IPC) gestionat în nucleu prin apelul `pipe`, descriptori de fișiere (*file descriptors*) și redirectarea fluxurilor de date (`dup`, `close`).

3. **Comparație între shell-ul xv6 și shell-ul Linux:** Shell-ul din xv6 este o variantă minimală didactică ce asigură facilitățile fundamentale (execuție, redirectări I/O și pipes), lipsindu-i funcționalitățile moderne din Bash precum istoricul comenzilor, autocompletarea prin tasta Tab sau variabilele de mediu.

4 Part 3: Analiza Codului Sursă

Sarcina Part 3: Read the source

După ieșirea din QEMU se deschide arborele sursă. Sistemul este împărțit în `kernel/` (sistemul de operare propriu-zis, rulat privilegiat: planificator, memorie, sistem de fișiere, handler-e de apeluri de sistem) și `user/` (programele obișnuite: `ls`, `cat`, `echo`, shell-ul).

2. Citirea unui program utilizator: `sed -n "1,40p" user/ls.c` și `sed -n "1,30p" user/cat.c`.

3. Identificarea apelurilor de sistem: `read()` și `write()` din `user/cat.c` sunt cereri către `kernel/`; se găsesc declarațiile lor (`cat user/user.h`) și implementările (`grep -n "sys_read\|sys_write" kernel/sysfile.c | head`).

Observe: (1) Ce apeluri de sistem folosește `user/cat.c` și ce cere fiecare nucleului? (2) În ce fișier al nucleului și la ce linie este implementat `sys_read`? (3) Într-o propoziție: care este diferența dintre codul din `kernel/` și cel din `user/`?

4.1 Inspectarea apelurilor de sistem

Pentru analiza modului în care programele din spațiul utilizator invocă nucleul, s-au executat următoarele comenzi în directorul gazdă `~/xv6-riscv`:

```
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

Output-ul pentru localizarea funcțiilor a returnat:

```
69:sys_read(void)
83:sys_write(void)
```

4.2 Observații (Part 3)

1. **Apeluri de sistem utilizate în `user/cat.c`:**
   - `read()`: Solicită nucleului să citească un flux de octeți dintr-un descriptor de fișier într-un buffer din memoria procesului.
   - `write()`: Solicită nucleului să transfere octeții dintr-un buffer către un descriptor de fișier (ex. descriptorul 1 pentru *stdout*).
   - `exit()`: Notifică nucleul despre terminarea execuției procesului curent, eliberând structurile și resursele asociate.
2. **Implementarea funcției `sys_read`:** Funcția este implementată în fișierul `kernel/sysfile.c`, începând cu linia **69**.
3. **Diferența dintre codul din `kernel/` și `user/`:** Codul din `kernel/` se execută în modul privilegiat (*supervisor mode*) cu acces direct la registrele hardware și memoria fizică, în timp ce codul din `user/` rulează în mod neprivilegiat (*user mode*), interacțiunea cu resursele fiind mediată strict prin apeluri de sistem.

5 Part 4: Implementarea Utilitarului `sleep`

Sarcina Part 4: Write your first xv6 program

Adăugarea unui program utilizator nou, `sleep`, care se oprește pentru un număr de tick-uri de ceas apelând apelul de sistem `sleep` (deja existent în nucleu).

4. Crearea fișierului `user/sleep.c`, cu verificarea argumentelor și afișarea mesajului de utilizare.
5. Adăugarea programului în lista `UPROGS` din `Makefile` (`$U/_sleep\`), păstrând indentarea cu tab și backslash-ul de la final.
6. Recompilarea, pornirea și testarea în xv6: `sleep 10` trebuie să facă o pauză și să revină la prompt, iar `sleep` fără argumente trebuie să afișeze mesajul de utilizare.

5.1 Codul sursă: `user/sleep.c`

A fost creat programul utilitar `sleep`, care primește numărul de tick-uri de ceas ca argument și apelează apelul de sistem existent:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
```

```
12    exit(0);
13  }
```

5.2 Configurarea build-ului în `Makefile`

În fișierul `Makefile`, binarul a fost adăugat în lista `UPROGS`:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    $U/_forktest\
```

5.3 Testarea programului

În consola xv6, testarea comportamentului s-a efectuat prin comenzile:

```
$ sleep 10
$ sleep
usage: sleep <ticks>
```

Comanda `sleep 10` a suspendat execuția pentru durata specificată, iar invocarea fără parametri a afișat corect mesajul de utilizare.

6 Concluzii

În cadrul lucrării a fost compilat și rulat nucleul xv6 în QEMU, au fost explorate utilitarele de bază și mecanismul de pipes, s-a analizat traseul unui apel de sistem de la programul utilizator (`user/cat.c`) până la implementarea sa în nucleu (`kernel/sysfile.c`) și s-a evidențiat separarea dintre modul utilizator și modul supervizor. Implementarea utilitarului `sleep` a demonstrat modul în care un program nou este integrat în sistem prin `Makefile` și cum acesta folosește un apel de sistem existent.